

1. 8 Puzzle Using DFS

```
def dfs(start, goal):
    stack = [(start, [])]
    visited = set()

    while stack:
        state, path = stack.pop()

        if state == goal:
            return path + [state]

        if state in visited:
            continue

        visited.add(state)

        zero = state.index(0)
        row, col = divmod(zero, 3)

        moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

        for dr, dc in moves:
            nr, nc = row + dr, col + dc

            if 0 <= nr < 3 and 0 <= nc < 3:
                new_zero = nr * 3 + nc
                new_state = list(state)

                new_state[zero], new_state[new_zero] = \
                    new_state[new_zero], new_state[zero]

                new_state = tuple(new_state)

                if new_state not in visited:
                    stack.append((new_state, path + [state]))

    return None
```

```

# User input
print("Enter initial state (use 0 for blank):")
start = tuple(map(int, input().split()))

# Goal state
goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

solution = dfs(start, goal)

if solution:
    print("\nSolution:")
    for state in solution:
        for i in range(0, 9, 3):
            print(state[i:i+3])
        print()
else:
    print("No solution found")

```

2. 8 Puzzle Using IDS

```

def dls(state, goal, depth, path, visited):
    if state == goal:
        return path

    if depth == 0:
        return None

    zero = state.index(0)
    row, col = divmod(zero, 3)

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dr, dc in moves:
        nr, nc = row + dr, col + dc

        if 0 <= nr < 3 and 0 <= nc < 3:
            new_zero = nr * 3 + nc

```

```

        new_state = list(state)

        new_state[zero], new_state[new_zero] = \
            new_state[new_zero], new_state[zero]

        new_state = tuple(new_state)

        if new_state not in visited:
            visited.add(new_state)

            result = dls(
                new_state,
                goal,
                depth - 1,
                path + [new_state],
                visited
            )

            if result:
                return result

    return None

def ids(start, goal):
    depth = 0

    while True:
        visited = {start}

        result = dls(
            start,
            goal,
            depth,
            [start],
            visited
        )

        if result:
            return result

```

```

        depth += 1

# User input
print("Enter initial state (use 0 for blank):")
start = tuple(map(int, input().split()))

# Goal state
goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

solution = ids(start, goal)

print("\nSolution:")

for state in solution:
    for i in range(0, 9, 3):
        print(state[i:i+3])
    print()

```

Output:

```

PS C:\Users\student\Desktop\1WA24CS205> python -u "c:\Users\student\Desktop\1WA24CS205\8PuzzleUsingIDS.py"
Enter initial state (use 0 for blank):
1 2 3 4 0 6 7 5 8

Solution:
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

PS C:\Users\student\Desktop\1WA24CS205> 

```